



**More Than a Bank
Your Financial Vault**

CIICC VAULT!

JAVA PROGRAMMING CLASS

INSTRUCTOR: MARCO YIMYAEM



KENNETH M. SALVADOR

BATCH 7 | YEAR 2025

OCTOBER 7, 2025

DESCRIPTION:

Ciicc Vault is a school project I developed to simulate the core features of a modern digital banking system. The website is designed to provide users with a secure and user-friendly platform for managing their finances online.

TECHNOLOGY USED:

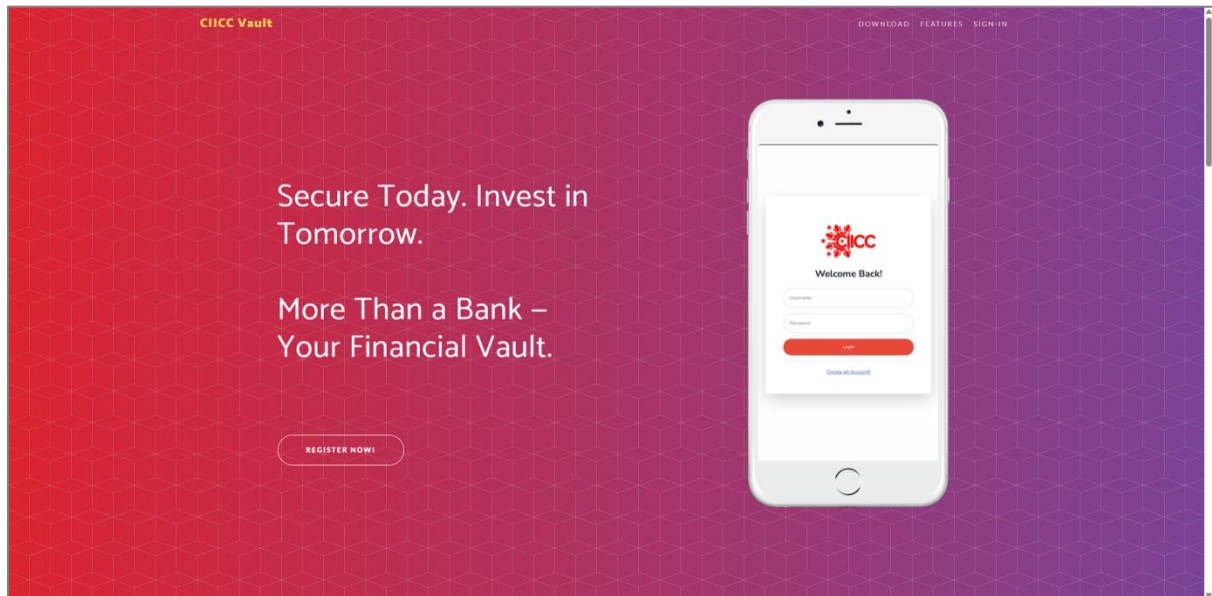
- **IntelliJ IDEA Community Edition** - for writing and managing Java code
- **XAMPP** – to run MySQL locally
- **phpMyAdmin** – to view and manage database easily through a web interface
- **HTML, CSS & Javascript** – for Graphic-User-Interface
- **Bootstrap** - framework to enhance the design of the web
- **Fetch API** – to communicate between web and java
- **Bouncy Castle & Salt Hash** - to encrypt and secure the password
- **Spring Web MVC** - for creating server-side HTML applications and REST APIs, following the Model-View-Controller design pattern.

OVERVIEW

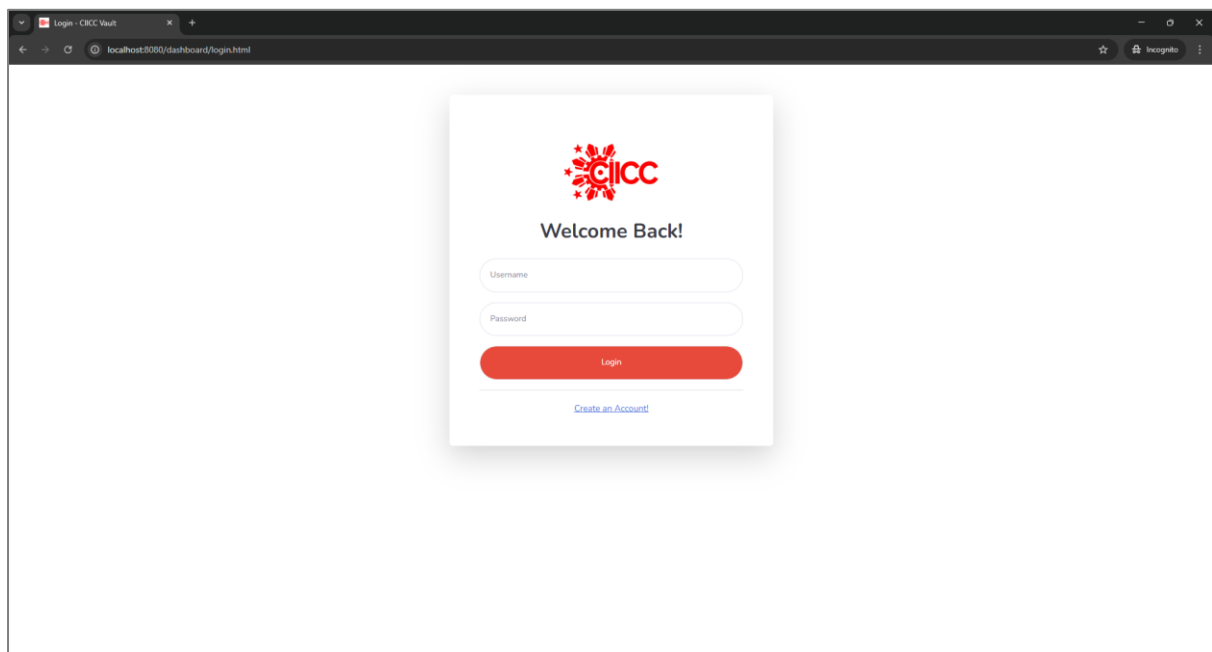
The CIICC Vault Web App is divided into several main components that designed to provide users with a secure and user-friendly platform for managing their finances online.

- **Home Page** - serving as the first page a visitor sees upon entering the website application
- **Login Page** – serves as the entry point for users to securely access their personal account or dashboard.
- **Registration Page** – allows new users to create an account by providing essential information to access the web app.
- **Dashboard Page** – is the central hub where users can view and interact with key information and Send Money within the web app.
- **Transaction History Page** - provides users with a detailed and organized view of all their past financial transactions within the web app. This page allows users to track, review, and manage their financial interactions with ease.
- **Master Account Page** – serves as the administrative or central control hub for managing and overseeing the overall account structure within the web app. This page provides users, typically admins or account owners, with comprehensive tools to manage sub-accounts and account settings at a higher level.

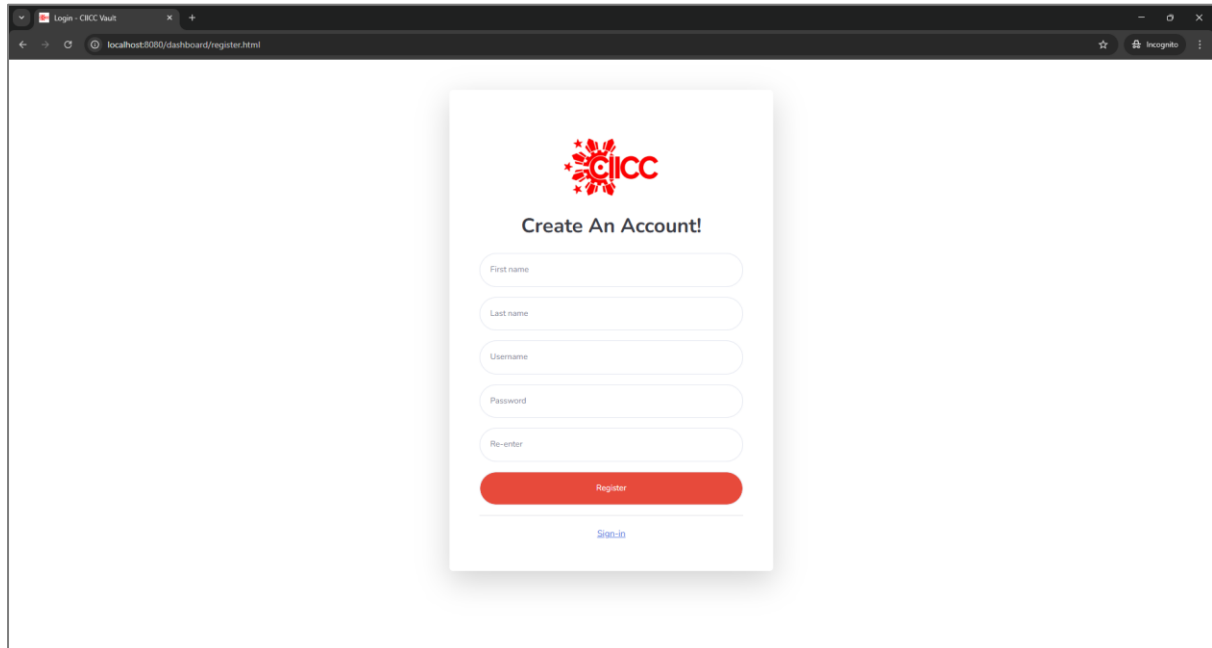
Home Page



Login



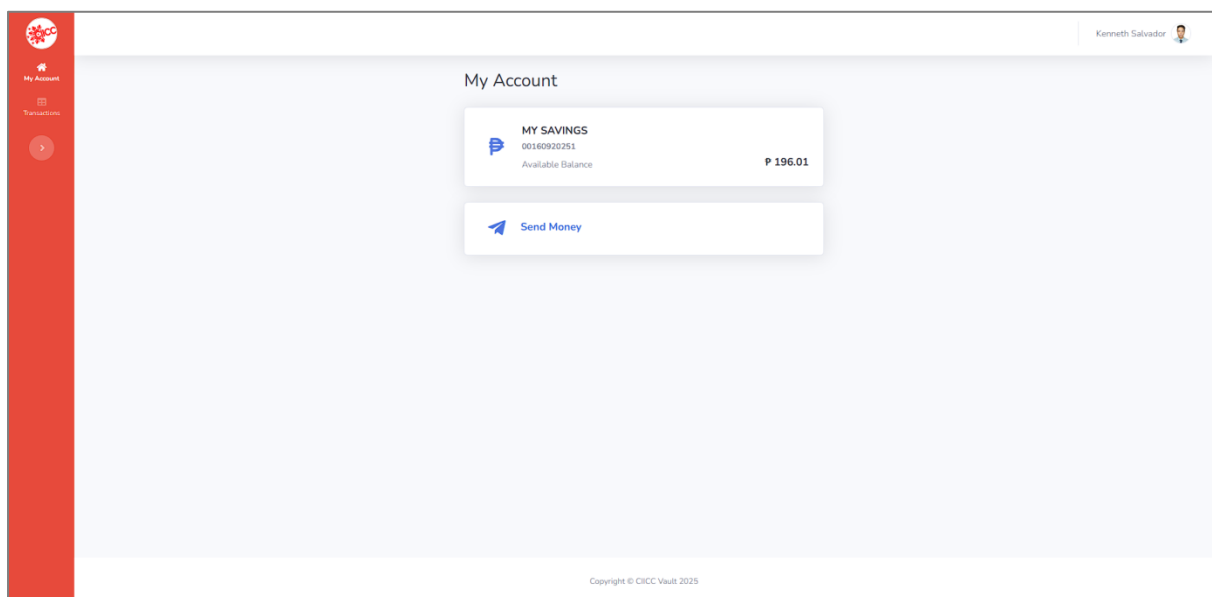
Registration



A screenshot of a web browser showing a registration form for 'CICC Vault'. The browser's address bar displays 'localhost:8080/dashboard/register.html'. The form is centered on a white background and features the CICC logo at the top. Below the logo, the text 'Create An Account!' is displayed. The form includes five input fields: 'First name', 'Last name', 'Username', 'Password', and 'Re-enter'. A red 'Register' button is positioned below these fields, and a 'Sign-In' link is located at the bottom of the form.

Register

Dashboard



A screenshot of a web dashboard for 'CICC Vault'. The dashboard has a red sidebar on the left with a 'My Account' section. The main content area is titled 'My Account' and displays two cards: 'MY SAVINGS' and 'Send Money'. The 'MY SAVINGS' card shows an available balance of P 196.01. The 'Send Money' card is below it. The user's name, 'Kenneth Salvador', is visible in the top right corner. The footer indicates 'Copyright © CICC Vault 2025'.

My Account

MY SAVINGS
00160920251
Available Balance P 196.01

Send Money

Kenneth Salvador

Copyright © CICC Vault 2025

Transaction History



My Account

Transactions

>

Kenneth Salvador

Transactions

History



Send Money

Reference: 4B4469771

To: 170920252

Sep 20, 2025

₡ 1.00



Deposit Money

Reference: 676022614

From: CIICC Vault-QC

Sep 19, 2025

₡ 0.01



Deposit Money

Reference: 653146065

From: CIICC Vault-QC

Sep 19, 2025

₡ 100.00



Send Money

Reference: 698911639

To: 170920252

Sep 18, 2025

₡ 2.00



Received Money

Reference: 910171744

From: 170920252

Sep 18, 2025

₡ 1.00

6 Results

<

1

2

>

Copyright © CIICC Vault 2025

Profile



My Account

Transactions

>

Kenneth Salvador

Profile



Kenneth Salvador

admin

[Change Password](#)

Copyright © CIICC Vault 2025

Master Account

The screenshot shows the Master Account web application. The header includes a logo and a 'Master Account' link. The main content area is titled 'Welcome Master Account'. On the left, there is a 'List of Accounts' section with a search bar and a table of accounts. On the right, there are two summary cards: 'ALL SAVINGS' showing a balance of P 2462.61 and a 'Deposit Money' button.

Status	Name	Account Number	Date Created	Available Balance	Actions
Enabled	Kenneth Salvador	160920251	Sep 16, 2025	150.64	Transactions Edit Delete
Enabled	Sir Marco	170920252	Sep 17, 2025	211.97	Transactions Edit Delete
Enabled	Manny Diaz	300920254	Sep 30, 2025	100.00	Transactions Edit Delete
Enabled	Ivan Toledo	300920255	Sep 30, 2025	10999.00	Transactions Edit Delete
Enabled	John Iverson Santos	300920256	Sep 30, 2025	-8999.00	Transactions Edit Delete

Copyright © CICC Vault 2025

Snap code of Bouncy Castle with Salt Hash

The screenshot shows an IDE with the PasswordHashers.java file open. The code implements password hashing and verification using Bouncy Castle's Argon2 parameters. It includes methods for hashing a password with a salt, generating a salt, and verifying a password and username.

```
package com.cliccvault.CliccVault;

import org.bouncycastle.crypto.params.Argon2Parameters;
import org.bouncycastle.crypto.generators.Argon2BytesGenerator;
import java.security.SecureRandom;
import java.util.Base64;

public class PasswordHashers {

    public static final int SALT_LENGTH = 16;
    public static final int HASH_LENGTH = 32;

    public static String hashPassword(char[] password, byte[] salt) {
        Argon2Parameters.Builder builder = new Argon2Parameters.Builder(Argon2Parameters.ARGON2_id)
            .withSalt(salt)
            .withParallelism(1)
            .withMemoryAsKB(1024) // 64 MB
            .withIterations(1);

        Argon2BytesGenerator generator = new Argon2BytesGenerator();
        generator.init(builder.build());

        byte[] hash = new byte[HASH_LENGTH];
        generator.generateBytes(password, hash);

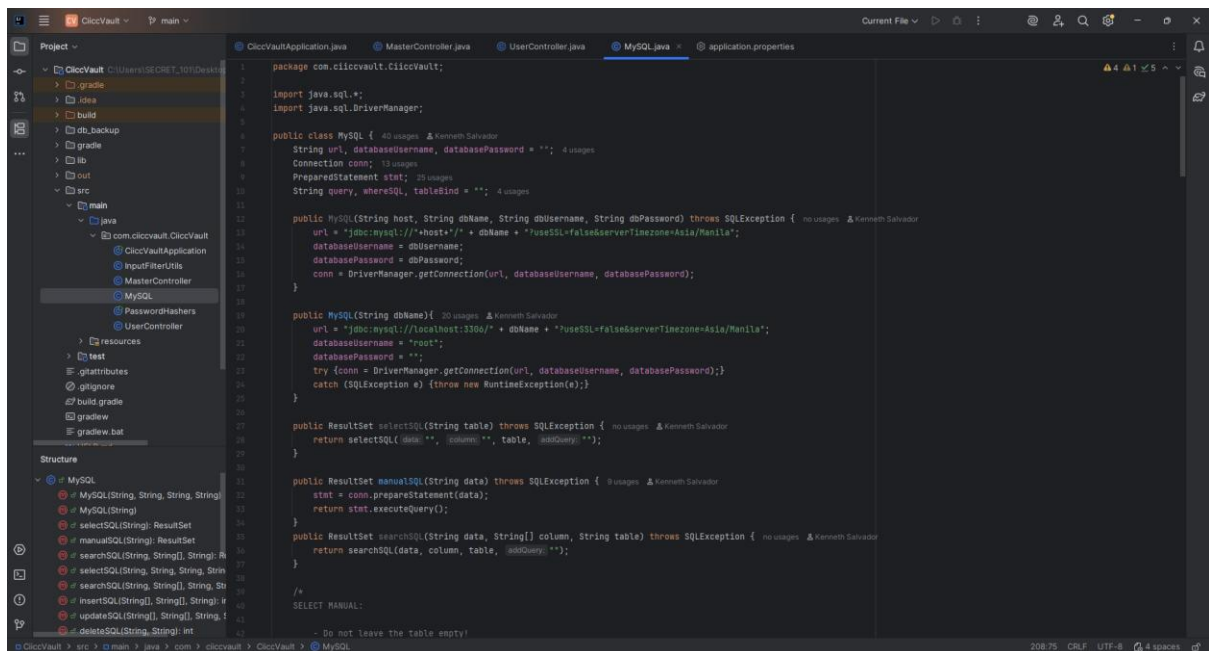
        return Base64.getEncoder().encodeToString(hash);
    }

    public static byte[] generateSalt() {
        byte[] salt = new byte[SALT_LENGTH];
        new SecureRandom().nextBytes(salt);
        return salt;
    }

    public static boolean verifyPassword(String inputPassword, String storedHash, String salt) {
        byte[] saltFromDB = Base64.getDecoder().decode(salt); //In the String is salt from DB
        String inputHash = hashPassword(inputPassword.toCharArray(), saltFromDB);
        return storedHash.equals(inputHash);
    }

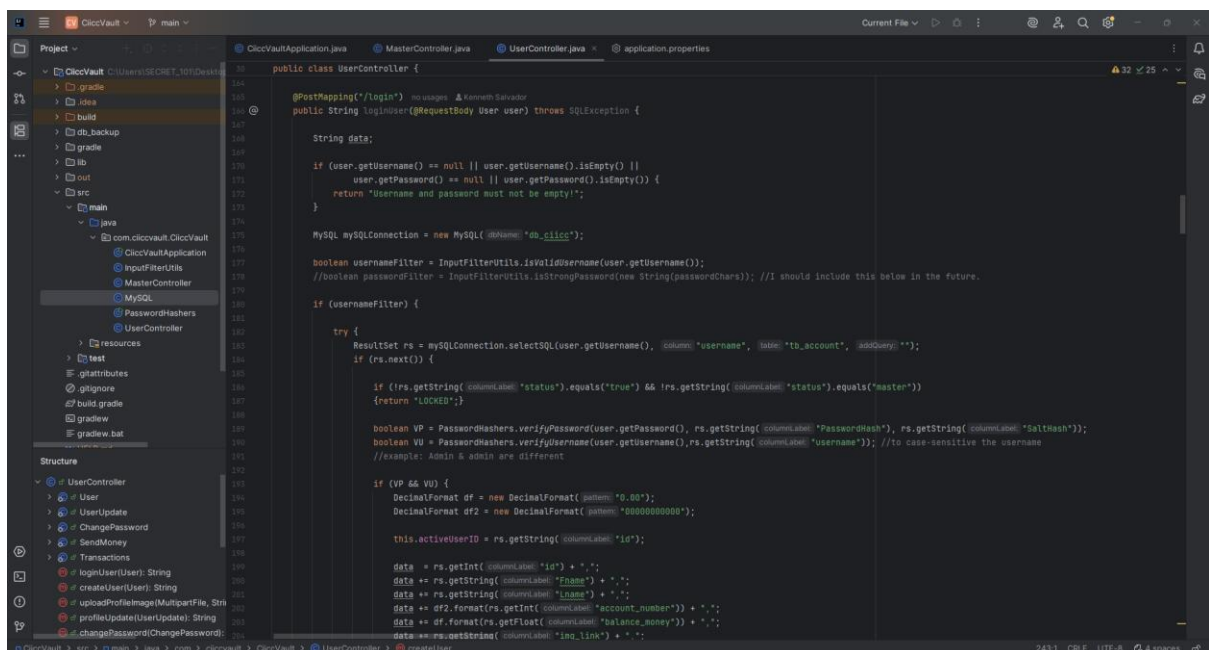
    public static boolean verifyUsername(String usernameInput, String usernameDB) {
        return usernameDB.equals(usernameInput);
    }
}
```

Snap code of my own made SQL Builder



```
1 package com.ciccvault.CiccVault;
2
3 import java.sql.*;
4 import java.sql.DriverManager;
5
6 public class MySQL {
7     String url, databaseUsername, databasePassword = "";
8     Connection conn;
9     PreparedStatement stat;
10    String query, whereSQL, tableBind = "";
11
12    public MySQL(String host, String dbName, String dbUsername, String dbPassword) throws SQLException {
13        url = "jdbc:mysql://" + host + "/" + dbName + "?useSSL=false&serverTimezone=Asia/Manila";
14        databaseUsername = dbUsername;
15        databasePassword = dbPassword;
16        conn = DriverManager.getConnection(url, databaseUsername, databasePassword);
17    }
18
19    public MySQL(String dbName) {
20        url = "jdbc:mysql://localhost:3306/" + dbName + "?useSSL=false&serverTimezone=Asia/Manila";
21        databaseUsername = "root";
22        databasePassword = "";
23        try {
24            conn = DriverManager.getConnection(url, databaseUsername, databasePassword);
25        } catch (SQLException e) {
26            throw new RuntimeException(e);
27        }
28    }
29
30    public ResultSet selectSQL(String table) throws SQLException {
31        return selectSQL("select * from " + table + " where 1=1");
32    }
33
34    public ResultSet manualSQL(String data) throws SQLException {
35        stat = conn.prepareStatement(data);
36        return stat.executeQuery();
37    }
38
39    public ResultSet searchSQL(String data, String[] column, String table) throws SQLException {
40        return searchSQL(data, column, table, "where 1=1");
41    }
42
43    /*
44     * SELECT MANUAL:
45     *
46     * Do not leave the table empty!
47     */
48}
```

Snap code of User-Controller with Function of Login



```
1 public class UserController {
2
3     @PostMapping("/login")
4     public String loginUser(@RequestBody User user) throws SQLException {
5
6         String data;
7
8         if (user.getUserName() == null || user.getUserName().isEmpty() ||
9             user.getPassword() == null || user.getPassword().isEmpty()) {
10            return "Username and password must not be empty!";
11        }
12
13        MySQL mySQLConnection = new MySQL("db_cicc");
14
15        boolean usernameFilter = InputFilterUtils.isValidUsername(user.getUserName());
16        //boolean passwordFilter = InputFilterUtils.isStrongPassword(new String(passwordChars)); //I should include this below in the future.
17
18        if (usernameFilter) {
19            try {
20                ResultSet rs = mySQLConnection.selectSQL("select * from " + "tbl_account" + " where " + "username" + " = '" + user.getUserName() + "'");
21                if (rs.next()) {
22                    if (rs.getString("status").equals("true") && rs.getString("password").equals("true")) {
23                        return "LOCKED!";
24                    }
25
26                    boolean VP = PasswordHashers.verifyPassword(user.getPassword(), rs.getString("password"));
27                    boolean VU = PasswordHashers.verifyUsername(user.getUserName(), rs.getString("username")); //to case-sensitive the username
28                    //example: Admin & admin are different
29
30                    if (VP && VU) {
31                        DecimalFormat df = new DecimalFormat("#.00");
32                        DecimalFormat df2 = new DecimalFormat("#0000000000");
33
34                        this.activeUserID = rs.getString("id");
35
36                        data = rs.getString("id") + ",";
37                        data += rs.getString("fname") + ",";
38                        data += rs.getString("lname") + ",";
39                        data += df2.format(rs.getInt("account_number")) + ",";
40                        data += df.format(rs.getFloat("balance_money")) + ",";
41                        data += rs.getString("link") + ",";
42                    }
43                }
44            } catch (SQLException e) {
45                e.printStackTrace();
46            }
47        }
48    }
49}
```

Source code uploaded in Github:



<https://github.com/shiene1995/CiiccVault>

Contact me.

I'm always open to collaborating on innovative and impactful projects. If you have an idea in mind or simply want to connect, I'd be glad to hear from you. Feel free to reach out anytime.



<https://www.linkedin.com/in/kenneth-salvador-9a920216b/>



<https://web.facebook.com/shiene1995>